

Execution Context, Call Stack, and Scope Chain in JavaScript

These are some of the most important JavaScript concepts asked in interviews. They explain **how JavaScript executes code, keeps track of variables, and manages function calls.**

Think of JavaScript as a worker following instructions one by one.

1. Execution Context

What is an Execution Context?

An **Execution Context** is the environment where JavaScript code is executed.

Whenever JavaScript runs code, it creates an execution context that contains:

1. Variables
2. Functions
3. The value of `this`
4. Information about the outer environment

Think of it as a **box** containing everything JavaScript needs to execute code.

Types of Execution Context

1. Global Execution Context (GEC)

Created when the JavaScript file starts running.

```
let name = "Gaurav";

function greet() {
  console.log("Hello");
}
```

JavaScript first creates a Global Execution Context.

It stores:

Global Execution Context

```
-----  
name = "Gaurav"  
greet = function  
this = window (browser)
```

Only one Global Execution Context exists.

2. Function Execution Context (FEC)

Created every time a function is called.

Example:

```
function greet() {  
  let message = "Hello";  
  console.log(message);  
}  
  
greet();
```

When `greet()` runs:

```
Global Context  
  ↓  
Function Context (greet)
```

The function gets its own execution context.

How Execution Context is Created

Each execution context is created in **2 phases**.

Phase 1: Memory Creation Phase

JavaScript scans the code first.

Variables and functions are stored in memory.

Example:

```
console.log(a);

var a = 10;

function greet() {
  console.log("Hello");
}
```

Memory Phase:

```
a = undefined

greet = function() {...}
```

Nothing is executed yet.

Phase 2: Execution Phase

Now JavaScript executes line by line.

```
console.log(a); // undefined

a = 10;
```

Final Memory:

```
a = 10
```

Execution Context Example

```
var x = 10;

function square(num) {
  var result = num * num;
  return result;
}

var output = square(x);
```

Step 1: Global Memory Phase

```
x = undefined

square = function
```

```
output = undefined
```

Step 2: Global Execution Phase

```
x = 10;
```

Memory:

```
x = 10
```

```
square = function
```

```
output = undefined
```

Function call:

```
square(x);
```

A new Function Execution Context is created.

Function Memory Phase

```
num = undefined
```

```
result = undefined
```

Function Execution Phase

```
num = 10
```

```
result = 100
```

Returns:

```
100
```

Global Memory becomes:

```
output = 100
```

2. Call Stack

What is Call Stack?

The **Call Stack** keeps track of which function is currently running.

Think of it like a stack of plates.

Rule:

- Last plate added → First plate removed
- LIFO = Last In First Out

Example

```
function one() {  
  two();  
}  
  
function two() {  
  three();  
}  
  
function three() {  
  console.log("Hello");  
}  
  
one();
```

Step 1

Global Context enters stack.

Stack
Global

Step 2

one() is called.

Stack
one()
Global

Step 3

`two()` is called.

Stack

```
two()  
one()  
Global
```

Step 4

`three()` is called.

Stack

```
three()  
two()  
one()  
Global
```

Step 5

`three()` finishes.

Stack

```
two()  
one()  
Global
```

Step 6

`two()` finishes.

Stack

```
one()  
Global
```

Step 7

`one()` finishes.

Stack

Global

Step 8

Program ends.

Stack

(empty)

Visual Representation

Start

Global

↓

one()

↓

two()

↓

three()

After completion

three() removed

two() removed

one() removed

Global removed

Call Stack Error

Example:

```
function hello() {  
  hello();  
}
```

```
hello();
```

Output:

```
RangeError:  
Maximum call stack size exceeded
```

Reason:

Functions keep getting pushed into the stack forever.

3. Scope

What is Scope?

Scope determines where a variable can be accessed.

Think of it as the visibility area of a variable.

Global Scope

```
let name = "Gaurav";

function show() {
  console.log(name);
}

show();
```

Output:

Gaurav

Global variables can be accessed anywhere.

Function Scope

```
function test() {
  let age = 25;
}

console.log(age);
```

Output:

ReferenceError

Because `age` only exists inside the function.

Block Scope

Variables declared with `let` and `const` are block-scoped.

```
if (true) {  
  let city = "Mumbai";  
}
```

```
console.log(city);
```

Output:

```
ReferenceError
```

var is NOT block scoped

```
if (true) {  
  var city = "Mumbai";  
}
```

```
console.log(city);
```

Output:

```
Mumbai
```

4. Scope Chain

What is Scope Chain?

When JavaScript cannot find a variable in the current scope, it searches in the outer scope.

This searching process is called the **Scope Chain**.

Example

```
let country = "India";
```

```
function outer() {  
  let state = "Maharashtra";  
  
  function inner() {  
    let city = "Panvel";  
  
    console.log(city);  
    console.log(state);  
    console.log(country);  
  }  
  
  inner();  
}  
  
outer();
```

Output:

```
Panvel  
Maharashtra  
India
```

How JavaScript Searches

Inside `inner()`:

```
console.log(country);
```

JavaScript searches:

1. inner scope?
✗ Not found
 2. outer scope?
✗ Not found
 3. global scope?
☑ Found
-

Scope Chain Visualization

```
Global Scope  
  country
```

↑

```
outer()  
  state
```

↑

```
inner()  
  city
```

The arrows represent the scope chain.

Real Interview Example

```
let a = 10;  
  
function outer() {  
  let b = 20;  
  
  function inner() {  
    let c = 30;  
  
    console.log(a);  
    console.log(b);  
    console.log(c);  
  }  
  
  inner();  
}  
  
outer();
```

Output:

```
10  
20  
30
```

Why?

```
inner() can access:  
✓ Its own variables  
✓ Parent variables  
✓ Global variables
```

Putting Everything Together

```
let message = "Hello";  
  
function first() {  
  let a = 10;  
  
  function second() {
```

```
    let b = 20;

    console.log(message);
    console.log(a);
    console.log(b);
  }

  second();
}

first();
```

Execution Flow

1. Global Execution Context created.
2. `first()` pushed into Call Stack.
3. Function Execution Context for `first()` created.
4. `second()` pushed into Call Stack.
5. Function Execution Context for `second()` created.
6. `second()` accesses variables through the Scope Chain.
7. `second()` removed from stack.
8. `first()` removed from stack.
9. Global Context removed.

Output:

```
Hello
10
20
```

Interview Definitions (1-Line Answers)

What is Execution Context?

An environment where JavaScript code is executed and variables/functions are stored.

What is Global Execution Context?

The execution context created when a JavaScript program starts.

What is Function Execution Context?

A new execution context created whenever a function is called.

What is Call Stack?

A data structure that keeps track of function execution order using LIFO.

What is Scope?

The area where a variable is accessible.

What is Scope Chain?

The mechanism JavaScript uses to look for variables in outer scopes when they are not found in the current scope.

Difference Between Scope and Scope Chain

Scope	Scope Chain
Defines where variables are accessible	Defines how JavaScript searches for variables
Visibility of variables	Lookup process
Created by blocks/functions	Connects nested scopes

Easy Memory Trick

Think of JavaScript execution like a company:

- **Execution Context** = Employee's workspace
- **Call Stack** = Queue of tasks
- **Scope** = Employee's access permissions
- **Scope Chain** = Asking higher managers when information isn't found locally

This analogy helps explain all three concepts together in interviews.

Data Types, Memory Allocation (Stack vs Heap), and Deep vs Shallow Copy in JavaScript

These are some of the **most important JavaScript interview topics**. If you understand them well, you'll find it much easier to learn variables, objects, arrays, functions, closures, and asynchronous JavaScript.

We'll learn everything step by step with simple examples.

1. What are Data Types?

A **data type** tells JavaScript **what kind of value** is stored inside a variable.

Example:

```
let name = "Gaurav";  
let age = 22;  
let isStudent = true;
```

Here,

- "Gaurav" → String
 - 22 → Number
 - true → Boolean
-

JavaScript has 2 categories of data types

```
Data Types  
├── Primitive Data Types  
└── Non-Primitive (Reference) Data Types
```

Primitive Data Types

Primitive values store **actual data directly**.

There are **7 primitive types**.

Type	Example
String	"Hello"
Number	100
Boolean	true
Undefined	undefined
Null	null
BigInt	1234567890n
Symbol	Symbol("id")

1. String

Text data.

```
let city = "Mumbai";
```

Output

Mumbai

2. Number

All numbers (integer and decimal).

```
let age = 25;  
let price = 99.99;
```

3. Boolean

Only two values.

```
let isLoggedIn = true;  
let isAdmin = false;
```

4. Undefined

Variable declared but not assigned.

```
let x;
```

```
console.log(x);
```

Output

```
undefined
```

5. Null

Represents an empty value intentionally.

```
let user = null;
```

Meaning:

"Currently there is no value."

Difference between Undefined and Null

```
let a;  
console.log(a);
```

Output

```
undefined
```

Because JavaScript hasn't assigned any value.

```
let b = null;  
  
console.log(b);
```

Output

```
null
```

Because **you intentionally assigned nothing**.

6. BigInt

Used for very large numbers.

```
let big = 123456789012345678901234567890n;
```

7. Symbol

Creates unique identifiers.

```
let id = Symbol("id");
```

Even if:

```
Symbol("id") === Symbol("id")
```

Output

```
false
```

Every Symbol is unique.

Non-Primitive (Reference) Data Types

These don't store the actual value directly in the variable. Instead, the variable stores a **reference (address)** to where the value lives in memory.

Examples:

```
let person = {  
  name: "Gaurav"  
};  
  
let arr = [1, 2, 3];  
  
function greet(){ }
```

Non-primitive types include:

- Objects
 - Arrays
 - Functions
 - Dates
 - Maps
 - Sets
-

Primitive vs Reference

Primitive

```
age = 22
```

Variable directly stores:

```
22
```

Reference

```
person = {name:"Gaurav"}
```

Variable stores

Address → Object

Instead of

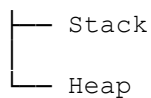
```
{name:"Gaurav"}
```

Memory Allocation

This is where **Stack and Heap** come into the picture.

JavaScript uses two main memory areas:

Memory



What is Stack Memory?

Stack stores

- Primitive values
- Function calls
- Local variables
- Execution contexts

Stack is:

- Fast
- Organized
- Small

Think of plates stacked one above another.

```
-----  
| Plate |  
-----  
| Plate |  
-----  
| Plate |  
-----
```

Last plate added

↓

First removed

This is called

LIFO

Last In First Out.

Example

```
let age = 22;  
let score = 90;
```

Stack

age → 22

score → 90

Simple.

What is Heap Memory?

Heap stores

- Objects
- Arrays
- Functions

Heap is

- Large
- Dynamic
- Slower than Stack

Imagine a huge warehouse.

Warehouse

Object A

Object B

Array

Function

Variables don't store the actual object.

They store its address.

Example

```
let person = {  
  name: "Gaurav"  
};
```

Memory

Stack

person

↓

0x100

Heap

0x100

```
{  
  name: "Gaurav"  
}
```

Variable stores

0x100

not the object itself.

Another Example

```
let arr = [10, 20, 30];
```

Stack

arr

↓

0x200

Heap

0x200

[10, 20, 30]

Stack vs Heap

Stack	Heap
Stores primitives	Stores objects
Small	Large
Faster	Slightly slower
Stores actual primitive values	Stores reference values
LIFO	Dynamic memory

Assignment of Primitive Values

```
let a = 10;
```

```
let b = a;  
  
b = 20;  
  
console.log(a);  
console.log(b);
```

Output

```
10  
  
20
```

Why?

Because primitive values are copied.

Memory

```
a → 10  
  
b → 10
```

Changing

```
b = 20
```

does not affect

```
a
```

Assignment of Objects

```
let person1 = {  
  name: "Gaurav"  
};  
  
let person2 = person1;
```

Memory

Stack

```
person1  
  
↓  
  
0x100
```

person2

↓

0x100

Heap

0x100

```
{  
  name: "Gaurav"  
}
```

Both point to SAME object.

Now

```
person2.name = "Rahul";
```

Object becomes

```
{  
  name: "Rahul"  
}
```

Now

```
console.log(person1.name);
```

Output

Rahul

Because both variables point to the same object.

Copying in JavaScript

There are two types.

Copy

└─ Shallow Copy

What is Shallow Copy?

A shallow copy copies **only the first level** of an object. If the object contains nested objects or arrays, those nested parts are still shared by reference.

Example:

```
let person = {
  name: "Gaurav",
  age: 22
};

let copy = {...person};

copy.name = "Rahul";

console.log(person.name);
console.log(copy.name);
```

Output

Gaurav

Rahul

Works fine because there are no nested objects.

Problem with Nested Objects

```
let person = {
  name: "Gaurav",
  address: {
    city: "Mumbai"
  }
};

let copy = {...person};

copy.address.city = "Pune";

console.log(person.address.city);
```

Output

Pune

Why?

Because only the top-level object was copied.

The nested object is shared.

Memory

person

↓

Object A

address

↓

Object B

copy

↓

Object C

address

↓

Object B

Both point to the same nested object.

Shallow Copy Methods

1. Spread Operator

```
let copy = {...obj};
```

2. Object.assign()

```
let copy = Object.assign({}, obj);
```

3. Array Spread

```
let arr = [1,2,3];  
let copy = [...arr];
```

4. slice()

```
let copy = arr.slice();
```

What is Deep Copy?

A deep copy creates **completely independent copies** of an object and all of its nested objects.

Example

```
let person = {  
  name: "Gaurav",  
  address: {  
    city: "Mumbai"  
  }  
};  
  
let copy = structuredClone(person);  
  
copy.address.city = "Pune";  
  
console.log(person.address.city);
```

Output

Mumbai

Because

person

↓

Object A

↓

Address A

copy

↓

Object B

↓

Address B

Everything is separate.

Deep Copy Methods

1. structuredClone() (Recommended)

```
let copy = structuredClone(obj);
```

Advantages

- Copies nested objects
 - Copies arrays
 - Fast
 - Modern JavaScript
-

2. JSON Method

```
let copy = JSON.parse(JSON.stringify(obj));
```

Works for simple objects.

Problems:

Cannot copy:

- Functions
- Undefined
- Symbol
- Date objects (they become strings)
- Map
- Set

Example

```
let obj = {
```

```
    date: new Date()
  };

let copy = JSON.parse(JSON.stringify(obj));
```

Now

Date

↓

String

Visual Difference

Shallow Copy

Original

↓

Object

↓

Nested Object

Copy

↓

New Object

↓

Same Nested Object

Nested object is shared.

Deep Copy

Original

↓

Object

↓

Nested Object

Copy

↓

New Object

↓

New Nested Object

Everything is copied independently.

Interview Questions

Q1. Are arrays primitive?

Answer: No. Arrays are objects (reference types).

```
typeof []
```

Output

```
"object"
```

Q2. Are functions objects?

Yes.

```
typeof function() {}
```

Output

```
"function"
```

Internally, functions are special objects.

Q3. Why does changing one object change another?

Because both variables point to the same object in heap memory.

```
let a = {x:1};
let b = a;

b.x = 10;

console.log(a.x);
```

Output

10

Q4. Which copy is faster?

- **Shallow copy** is generally faster because it copies only the top level.
 - **Deep copy** takes more time and memory because it recursively copies nested structures.
-

Q5. When should I use deep copy?

Use a deep copy when:

- You need to modify nested properties without affecting the original object.
 - You're working with application state (for example, in React or Redux).
 - You need a completely independent clone of complex data.
-

Quick Revision Table

Topic	Primitive	Reference
Stored in	Stack	Heap (with reference stored on the stack)
Examples	Number, String, Boolean, Null, Undefined, Symbol, BigInt	Object, Array, Function, Map, Set
Assignment	Copies the value	Copies the reference (memory address)
Comparison	Compared by value	Compared by reference
Changing copy affects original?	✗No	✓Yes (unless you create a new object)

Shallow Copy vs Deep Copy

Feature	Shallow Copy	Deep Copy
Copies top-level properties	☑ Yes	☑ Yes
Copies nested objects	✗ No (shares references)	☑ Yes (creates new nested objects)
Original changes when nested copy changes	☑ Yes	✗ No
Common methods	..., Object.assign(), slice()	structuredClone(), JSON.parse(JSON.stringify()) (limited)

Memory Trick to Remember

- **Primitive values** → Stored directly → **Stack** → **Copied by value**
- **Objects/Arrays/Functions** → Stored in **Heap** → Variables hold a **reference** → **Copied by reference**
- **Shallow copy** → New outer object, **same nested objects**
- **Deep copy** → Entirely new object, **new nested objects**

Understanding these concepts is essential because they explain why variables behave differently, why objects can change unexpectedly, and how JavaScript manages memory under the hood. They are among the most frequently asked topics in JavaScript interviews.

Closures and Lexical Scope in JavaScript (And How They Can Cause Memory Leaks)

These are among the **most important JavaScript interview topics** because they are used everywhere:

- Event listeners
- React Hooks
- Callbacks
- Timers
- Async programming
- Data privacy
- Modules

Many developers use closures every day **without realizing it**.

First Understand Scope

A **scope** is simply the area where a variable can be accessed.

There are mainly three types:

Global Scope
Function Scope
Block Scope

Example:

```
let globalVar = "I am global";

function test() {
  let functionVar = "I am inside function";

  if (true) {
    let blockVar = "I am inside block";
  }

  console.log(functionVar); // Works
  console.log(globalVar);   // Works

  console.log(blockVar);    // Error
}

test();
```

Output

```
I am inside function  
I am global  
ReferenceError
```

What is Lexical Scope?

This is the foundation of closures.

Definition

Lexical Scope means JavaScript determines variable accessibility based on where functions are written in the code (their physical location).

Think of it as:

A function remembers where it was created, not where it is called.

Example 1

```
let name = "Gaurav";  
  
function greet() {  
    console.log(name);  
}  
  
greet();
```

Output

```
Gaurav
```

Why?

When `greet()` looks for `name`

It searches

```
Inside greet()
```

↓

```
Outside greet()
```

↓

Global Scope

It finds `name`.

Example 2

```
function outer() {  
    let message = "Hello";  
  
    function inner() {  
        console.log(message);  
    }  
  
    inner();  
}  
  
outer();
```

Output

Hello

The `inner()` function can access variables of its parent.

Picture it:

Global

↓

`outer()`

↓

`inner()`

The search always moves **outward**, never inward.

Example 3

```
function outer() {
```

```
    let city = "Mumbai";

    function inner() {
        console.log(city);
    }

    return inner;
}

const result = outer();

result();
```

Output

Mumbai

Even though `outer()` finished executing...

`inner()` still remembers `city`.

This is called a **Closure**.

What is a Closure?

Definition

A **Closure** is a function that remembers the variables from its outer (lexical) scope even after the outer function has finished executing.

The official definition:

A closure is the combination of a function bundled together with references to its surrounding state.

Simple Real-Life Example

Imagine a backpack.

```
Outer Function
|
|--- Variable
```

```
|  
|--- Variable  
|  
|--- Inner Function
```

When the inner function leaves,
it takes a backpack containing all variables it needs.
Even if the outer function disappears,
the backpack remains.

Example 1 (Basic Closure)

```
function outer() {  
    let count = 0;  
  
    function inner() {  
        count++;  
        console.log(count);  
    }  
  
    return inner;  
}  
  
const counter = outer();  
  
counter();  
counter();  
counter();
```

Output

```
1  
2  
3
```

Notice:

`count` is **not recreated**.

It is remembered.

Memory Visualization

When calling

```
const counter = outer();
```

Memory becomes

Heap

```
counter
  ↓
inner function
  ↓
Closure
count = 0
```

When

```
counter();
count = 1
```

Again

```
count = 2
```

Again

```
count = 3
```

The variable stays alive.

Example 2 (Private Variables)

Closures allow data hiding.

```
function bankAccount() {
  let balance = 1000;

  return {
    deposit(amount) {
      balance += amount;
    },
    withdraw(amount) {
```

```
        balance -= amount;
    },

    showBalance() {
        console.log(balance);
    }

};

const account = bankAccount();

account.showBalance();

account.deposit(500);

account.showBalance();

console.log(account.balance);
```

Output

1000

1500

undefined

Why?

Because

balance

cannot be accessed directly.

Only the closure can access it.

This is called **Encapsulation**.

Example 3 (Function Factory)

Closures can create customized functions.

```
function multiplyBy(num) {

    return function(value) {
        return value * num;
    }
}
```

```
}  
  
const double = multiplyBy(2);  
const triple = multiplyBy(3);  
console.log(double(5));  
console.log(triple(5));
```

Output

10

15

Here

double

remembers

num = 2

while

triple

remembers

num = 3

Example 4 (Counter)

```
function createCounter() {  
    let count = 0;  
    return {  
        increment() {  
            count++;  
        },  
        decrement() {  
            count--;  
        },  
        getCount() {
```

```
        return count;
    }

    };
}

const counter = createCounter();

counter.increment();

counter.increment();

console.log(counter.getCount());
```

Output

2

Closures in Loops (Famous Interview Question)

Wrong Example

```
for (var i = 1; i <= 3; i++) {

    setTimeout(function () {
        console.log(i);
    }, 1000);

}
```

Output

4
4
4

Why?

Because

var

has function scope.

There is only one variable `i`.

After the loop

```
i = 4
```

Every closure uses the same variable.

Correct Solution (Using `let`)

```
for (let i = 1; i <= 3; i++) {  
    setTimeout(function () {  
        console.log(i);  
    }, 1000);  
}
```

Output

```
1  
2  
3
```

Because `let` creates a new variable for every iteration.

Another Solution (Using Closure)

```
for (var i = 1; i <= 3; i++) {  
    (function(x) {  
        setTimeout(function() {  
            console.log(x);  
        }, 1000);  
    })(i);  
}
```

Output

```
1  
2  
3
```

Each function gets its own copy of `x`.

Where Are Closures Used?

1. Event Listeners

```
function setupButton() {  
    let clicks = 0;  
    button.addEventListener("click", function() {  
        clicks++;  
        console.log(clicks);  
    });  
}
```

Each click remembers

clicks

2. Timers

```
function timer() {  
    let seconds = 0;  
    setInterval(function() {  
        seconds++;  
        console.log(seconds);  
    }, 1000);  
}
```

3. React

```
useState()  
useEffect()  
useCallback()
```

All rely on closures.

4. Callbacks

```
fetch(url)

.then(function(response) {

});
```

Callbacks often capture variables from their surrounding scope.

Advantages of Closures

☑ Data privacy

```
let password
```

cannot be modified directly.

☑ State preservation

Counters

Shopping carts

Game scores

Timers

☑ Function factories

```
double()
```

```
triple()
```

```
square()
```

☑Module pattern

Before ES6 modules, closures were widely used to hide internal variables and expose only selected functions.

Disadvantages of Closures

- Consume extra memory because captured variables remain alive.
 - Can accidentally keep large objects in memory.
 - Can create memory leaks if references are not released.
-

How Closures Cause Memory Leaks

First Understand Garbage Collection

JavaScript automatically frees memory that is **no longer reachable**.

For example:

```
function test() {  
    let arr = new Array(1000000);  
}
```

```
test();
```

After `test()` finishes:

```
arr
```

has no references.

The garbage collector removes it.

But Closures Keep Variables Alive

Example:

```
function outer() {
```

```
    let hugeArray = new Array(1000000).fill("JavaScript");

    return function () {
      console.log(hugeArray.length);
    };
  }

  const fn = outer();
```

Even though `outer()` has finished,

`hugeArray`

still exists.

Why?

Because the returned function still references it.

Memory looks like:

```
fn
|
▼
Closure
|
▼
hugeArray
```

Since `fn` exists, the array cannot be garbage-collected.

Real Memory Leak Example

```
function createUser() {

  let largeData = new Array(1000000).fill("User Data");

  return function () {
    console.log("Hello");
  };

}

let user = createUser();
```

Notice something?

The returned function never uses `largeData`.

But because `largeData` is part of the closure's lexical environment, some JavaScript engines may keep it alive as long as the closure exists (modern engines may optimize this away in some cases).

Another Common Memory Leak

```
const button = document.querySelector("button");

button.addEventListener("click", function () {
  console.log("Clicked");
});
```

If the button is removed from the page but the event listener isn't removed (or another object still references it), the closure and any captured variables may remain in memory longer than necessary.

Proper cleanup:

```
function handleClick() {
  console.log("Clicked");
}

button.addEventListener("click", handleClick);

// Later
button.removeEventListener("click", handleClick);
```

Timers Can Also Leak Memory

```
function startTimer() {

  let data = new Array(1000000);

  setInterval(function() {

    console.log(data.length);

  }, 1000);

}
```

Since the interval runs forever,

`data`

also stays alive.

Solution:

```
const id = setInterval(function() {  
  }, 1000);  
  
clearInterval(id);
```

Best Practices to Avoid Closure-Related Memory Leaks

☑ Capture only what you need

Instead of capturing a huge object:

```
let user = {  
  name: "Gaurav",  
  profile: new Array(1000000)  
};  
  
return function() {  
  console.log(user.name);  
};
```

Capture only the needed value:

```
let name = user.name;  
  
return function() {  
  console.log(name);  
};
```

☑ Remove event listeners when no longer needed

```
button.removeEventListener(...);
```

☑ Clear timers

```
clearTimeout(id);  
  
clearInterval(id);
```

☑ Release references when finished

```
let counter = createCounter();  
  
// Later  
counter = null;
```

Now, if nothing else references the closure, the garbage collector can reclaim its memory.

Interview Questions

1. What is lexical scope?

Answer: Lexical scope means a function can access variables from the scope where it was **defined**, not where it is called.

2. What is a closure?

Answer: A closure is a function that remembers and can access variables from its outer lexical scope even after the outer function has finished executing.

3. Why are closures useful?

- Data privacy
 - State preservation
 - Function factories
 - Callbacks
 - Event handlers
 - Module patterns
-

4. Can closures cause memory leaks?

Answer: Yes. If a closure continues to reference variables or large objects that are no longer needed, those objects cannot be garbage-collected until the closure itself becomes unreachable.

5. Difference between Lexical Scope and Closure

Lexical Scope	Closure
A language rule that determines where variables are accessible based on where code is written.	A runtime behavior where a function keeps access to variables from its lexical scope after the outer function has finished executing.
Exists whether or not a closure is created.	Depends on lexical scope.
Decides <i>where</i> variables can be found.	Remembers <i>which</i> variables should stay available.

Easy Way to Remember

- **Lexical Scope** = "Where was the function **written**?"
- **Closure** = "What variables does the function **remember** after the outer function has finished?"
- **Memory Leak** = "Did the closure keep data alive longer than necessary?"

Think of it like this:

Lexical Scope decides what a function *can access*. **Closure** preserves what a function *does access* after its outer function has returned.

Hoisting, Temporal Dead Zone (TDZ), and `var` VS `let` VS `const` (Complete Beginner's Guide)

These are among the **most frequently asked JavaScript interview topics** because they explain how JavaScript behaves **before your code even starts executing**.

We'll cover them step by step.

Table of Contents

1. What is Hoisting?
 2. How JavaScript Executes Code
 3. Hoisting with `var`
 4. Hoisting with `let`
 5. Hoisting with `const`
 6. What is the Temporal Dead Zone (TDZ)?
 7. Why TDZ Exists
 8. Hoisting of Functions
 9. `var` VS `let` VS `const`
 10. Memory Visualization
 11. Real-Life Analogy
 12. Interview Questions
 13. Common Mistakes
-

1. What is Hoisting?

Definition

Hoisting means JavaScript moves declarations to the top of their scope before executing the code.

Notice one important word:

Declarations are hoisted, NOT initializations.

Example:

```
console.log(name);
```

```
var name = "John";
```

It looks like:

```
var name;  
console.log(name);  
name = "John";
```

Output

```
undefined
```

Why?

Because only

```
var name;
```

was moved.

Not

```
name = "John";
```

2. How JavaScript Executes Code

Whenever a program starts, JavaScript does **two phases**.

Phase 1: Memory Creation Phase

JavaScript scans the whole file.

It allocates memory.

Example

```
var a = 10;  
let b = 20;  
const c = 30;
```

```
function hello() {}
```

Memory becomes

a → undefined

b → <uninitialized>

c → <uninitialized>

hello → entire function

Notice:

var

gets

undefined

while

let

and

const

remain

uninitialized

Phase 2: Execution Phase

Now JavaScript executes line by line.

```
var a = 10;
```

Memory

a → 10

Next

```
let b = 20;
```

Memory

`b → 20`

Next

`const c = 30;`

Memory

`c → 30`

3. Hoisting with `var`

Example

```
console.log(a);
```

```
var a = 5;
```

Memory phase

`a → undefined`

Execution

```
console.log(a);
```

prints

`undefined`

Then

```
a = 5;
```

Output

`undefined`

No error.

Another example

```
var x;
```

```
console.log(x);
```

Output

undefined

Because

`x` exists

but has no value.

4. Hoisting with `let`

Example

```
console.log(age);  
let age = 25;
```

Output

ReferenceError

Why?

People think

`let` isn't hoisted.

This is wrong.

Actually,

`let` **IS** hoisted.

But

it stays inside the

Temporal Dead Zone (TDZ).

So JavaScript doesn't allow access.

Memory phase

age → uninitialized

Execution

```
console.log(age);
```

JavaScript says

No.
You cannot use it yet.

Error

```
ReferenceError:  
Cannot access 'age' before initialization
```

5. Hoisting with `const`

Exactly same hoisting behavior.

Example

```
console.log(PI);  
  
const PI = 3.14;
```

Output

```
ReferenceError
```

Again

```
PI exists  
  
but  
  
is uninitialized.
```

6. What is the Temporal Dead Zone (TDZ)?

This is one of the most asked interview questions.

Definition

The **Temporal Dead Zone** is the time between

variable creation

and

variable initialization.

Example

```
console.log(a);
```

```
let a = 10;
```

Timeline

Program starts

↓

Memory created

↓

a exists

↓

a is uninitialized

↓

```
console.log(a)
```

↓

ERROR

↓

```
a = 10
```

↓

TDZ ends

Visual

Memory Phase

a → uninitialized

Execution Phase

console.log(a)

↑
ERROR

let a = 10

↑
Initialization

TDZ Ends

Another example

```
{  
  console.log(score);  
  let score = 50;  
}
```

Output

ReferenceError

After initialization

```
let score = 50;
```

```
console.log(score);
```

Output

50

No problem.

7. Why Does TDZ Exist?

Suppose JavaScript allowed this:

```
let age;
```

```
console.log(age);
```

before initialization.

You might accidentally use variables before assigning values.

TDZ prevents many bugs.

Example

Wrong

```
let total;  
price = total * 5;
```

If JavaScript silently allowed this, debugging would be difficult.

Instead it throws an error immediately.

8. Hoisting of Functions

Functions behave differently.

Function Declaration

```
hello();  
  
function hello() {  
    console.log("Hello");  
}
```

Output

Hello

Why?

Entire function is hoisted.

Memory

hello → complete function

Function Expression

```
sayHi();

var sayHi = function () {
  console.log("Hi");
};
```

Memory

sayHi → undefined

Execution

```
sayHi();
```

Actually becomes

```
undefined();
```

Output

```
TypeError:
sayHi is not a function
```

Arrow Function

```
greet();

const greet = () => {
  console.log("Hello");
};
```

Output

```
ReferenceError
```

Because

```
const
```

is inside TDZ.

9. `var` VS `let` VS `const`

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Hoisted	☑Yes	☑Yes	☑Yes
Initial value	undefined	Uninitialized	Uninitialized
TDZ	✗No	☑Yes	☑Yes
Reassign	☑Yes	☑Yes	✗No
Redeclare	☑Yes	✗No	✗No
Scope	Function	Block	Block
Must initialize	✗No	✗No	☑Yes

10. Memory Visualization

Suppose

```
var a = 10;  
let b = 20;  
const c = 30;
```

Memory Phase

Global Memory

```
a → undefined  
b → uninitialized  
c → uninitialized
```

Execution

Line 1

```
a → 10
```

Line 2

```
b → 20
```

Line 3

```
c → 30
```

Final Memory

a → 10

b → 20

c → 30

11. Real-Life Analogy

Imagine a hotel.

Before guests arrive:

Room 101 reserved

Room 102 reserved

Room 103 reserved

Memory creation phase.

var

Room

101

already has temporary furniture.

undefined

You can enter.

let

Room

102

is locked.

You cannot enter.

Until setup finishes.

Trying to enter

↓

Security stops you.

ReferenceError

const

Same as `let`

except

the furniture can only be placed once.

12. Interview Questions

Q1. What is hoisting?

Answer:

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope during the memory creation phase before execution. Only declarations are hoisted; initializations remain in place.

Q2. Is `let` hoisted?

Yes.

It is hoisted.

But it stays in the **Temporal Dead Zone** until initialized.

Q3. Why does `var` print `undefined`?

Because during the memory creation phase JavaScript initializes `var` variables with `undefined`.

Q4. Why do `let` and `const` throw `ReferenceError`?

Because they are hoisted but remain **uninitialized** until their declaration is executed. Accessing them before initialization is accessing them inside the **Temporal Dead Zone**.

Q5. What is the Temporal Dead Zone?

It is the period between a variable's creation (during the memory phase) and its initialization (during the execution phase), during which accessing the variable throws a `ReferenceError`.

Q6. Why does `const` require initialization?

Because a `const` variable cannot be reassigned later. JavaScript requires its value to be provided at the time of declaration.

```
const PI = 3.14;    // ✔  
const PI;           // ✗ SyntaxError
```

13. Common Mistakes

✗Mistake 1

```
console.log(a);  
let a = 5;
```

Error

`ReferenceError`

✗Mistake 2

```
const x;
```

```
x = 10;
```

Error

```
SyntaxError
```

✖ Mistake 3

```
let a = 5;
```

```
let a = 10;
```

Error

```
Identifier has already been declared
```

✖ Mistake 4

```
var a = 5;
```

```
var a = 10;
```

This is allowed.

Output

```
10
```

✖ Mistake 5

```
const person = {  
  name: "John"  
};
```

```
person.name = "Mike";
```

This works because the object itself isn't being reassigned—only one of its properties is being updated.

Output

```
{  
  name: "Mike"  
}
```

However:

```
const person = {  
  name: "John"  
};  
  
person = {};
```

Output

```
TypeError:  
Assignment to constant variable
```

Quick Revision Cheat Sheet

Topic	var	let	const
Hoisted	✓	✓	✓
Initialized during memory phase	undefined	✗	✗
Temporal Dead Zone	✗	✓	✓
Scope	Function	Block	Block
Can reassign	✓	✓	✗
Can redeclare	✓	✗	✗
Must initialize	✗	✗	✓

Key Takeaways

- **Hoisting** means JavaScript processes declarations before execution.
- `var` is hoisted and initialized to `undefined`, so accessing it before assignment prints `undefined`.
- `let` and `const` are also hoisted, but they remain **uninitialized** until their declaration is executed.
- The **Temporal Dead Zone (TDZ)** is the period between a variable's creation and initialization. Accessing a `let` or `const` variable during this period throws a `ReferenceError`.
- **Function declarations** are fully hoisted and can be called before they appear in the code.
- **Function expressions** and **arrow functions** are **not** callable before their assignment because only the variable declaration is hoisted.

Understanding these concepts will also make it much easier to learn **Execution Context**, **Scope Chain**, and **Closures**, since all of them depend on how JavaScript creates memory and executes code.

Prototypes, Prototypal Inheritance, and ES6 Classes Under the Hood

These are **one of the most important JavaScript interview topics** because JavaScript is **prototype-based**, not class-based.

Many people learn `class` syntax and think JavaScript works like Java or C++, but internally **everything still works using prototypes**.

By the end of this guide, you'll understand:

- ☒ What is a prototype?
- ☒ What is the prototype chain?
- ☒ How inheritance works
- ☒ How ES6 classes actually work internally
- ☒ Difference between class and prototype
- ☒ Interview questions
- ☒ Common mistakes

Before We Start

Imagine a classroom.

There are **100 students**.

Every student has:

- name
- age

But every student also knows how to:

- walk()
- study()
- eat()

Would JavaScript copy these functions 100 times?

No.

Instead:

JavaScript stores the functions **once** and lets every object use them.

That shared place is called the **Prototype**.

What is a Prototype?

A prototype is simply another object.

JavaScript objects can inherit properties and methods from another object.

Example:

```
const person = {  
  walk() {  
    console.log("Walking");  
  }  
};  
  
const student = {  
  name: "John"  
};
```

Currently,

```
student  
  
{  
  name: "John"  
}
```

No walk() function.

But suppose JavaScript links them:

```
student  
  |  
  |  
  ▼  
person
```

Now:

```
student.walk();
```

Output

Walking

Even though student doesn't have walk().

It borrowed it from its prototype.

Every Object Has a Prototype

Example

```
const obj = {};
```

Looks empty.

But internally:

```
obj
  |
  ▼
Object.prototype
```

Object.prototype contains methods like

```
toString()
```

```
hasOwnProperty()
```

```
valueOf()
```

```
isPrototypeOf()
```

```
etc.
```

That's why this works:

```
const obj = {};
```

```
console.log(obj.toString());
```

Output

```
"[object Object]"
```

You never created `toString()`.

It came from `Object.prototype`.

Prototype Chain

Suppose

```
const animal = {  
  eats: true  
};  
  
const dog = {  
  barks: true  
};
```

Link them.

```
Object.setPrototypeOf(dog, animal);
```

Now memory looks like

```
dog  
|  
▼  
animal  
|  
▼  
Object.prototype  
|  
▼  
null
```

This chain is called the **Prototype Chain**.

Property Lookup

When JavaScript cannot find a property,

it searches upward.

Example

```
const animal = {  
  eats: true  
};  
  
const dog = {  
  barks: true  
};
```

```
Object.setPrototypeOf(dog, animal);  
  
console.log(dog.eats);
```

How JS searches:

Step 1

dog.eats ?

Not found

↓

Step 2

animal.eats ?

Found!

Output

true

Another Example

```
const grandParent = {  
  money: 10000  
};  
  
const parent = {};  
  
const child = {};  
  
Object.setPrototypeOf(parent, grandParent);  
Object.setPrototypeOf(child, parent);  
  
console.log(child.money);
```

Search process

child

↓

parent

↓

grandParent

↓

Found money

Output

10000

If Property Doesn't Exist Anywhere

```
console.log(child.house);
```

Search

child

↓

parent

↓

grandParent

↓

Object.prototype

↓

null

Result

undefined

Shadowing

If child has same property,

JavaScript stops searching.

Example

```
const animal = {  
  legs: 4
```

```
};

const dog = {
  legs: 3
};

Object.setPrototypeOf(dog, animal);

console.log(dog.legs);
```

Output

3

Because own properties always win.

Constructor Functions

Before ES6,

people created objects like this.

```
function Person(name) {
  this.name = name;
}
```

Now create object

```
const p1 = new Person("John");
```

What happens internally?

Step 1

Creates empty object

```
{}
```

↓

Step 2

Links it to

```
Person.prototype
```

↓

Step 3

Calls constructor

```
this = new object
```

↓

Step 4

Returns object

Adding Methods

Instead of

```
function Person(name) {  
  this.name = name;  
  
  this.sayHello = function () {  
    console.log("Hello");  
  };  
}
```

This creates a new function for every object.

Bad.

Better:

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.sayHello = function () {  
  console.log("Hello");  
};
```

Now

Person.prototype

↓

sayHello()

Shared by every object.

Visual Memory

```
p1
  name
    ↓
  Person.prototype
    sayHello()
      ↓
      Object.prototype
        ↓
        null
```

Example

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log("Hello " + this.name);
};

const p1 = new Person("John");
const p2 = new Person("Alice");

p1.sayHello();
p2.sayHello();
```

Output

```
Hello John
Hello Alice
```

Only ONE copy of sayHello exists.

How new Keyword Works

```
const p = new Person("John");
```

Internally

```
Create {}
```

↓

```
Set prototype
```

↓

```
Call constructor
```

↓

```
Return object
```

Equivalent to

```
const obj = {};
```

```
Object.setPrototypeOf(obj, Person.prototype);
```

```
Person.call(obj, "John");
```

```
return obj;
```

Prototypal Inheritance

Suppose

```
function Animal() {}
```

Methods

```
Animal.prototype.eat = function() {  
  console.log("Eating");  
};
```

Now

```
function Dog() {}
```

Inheritance

```
Dog.prototype = Object.create(Animal.prototype);
```

Meaning

Dog.prototype

↓

Animal.prototype

Now add

```
Dog.prototype.bark = function() {  
    console.log("Woof");  
};
```

Create object

```
const d = new Dog();  
  
d.eat();  
  
d.bark();
```

Output

Eating

Woof

Prototype Chain

d

↓

Dog.prototype

↓

Animal.prototype

↓

Object.prototype

↓

null

ES6 Classes

Now modern syntax

```
class Person {  
    constructor(name) {  
        this.name = name;  
    }  
  
    sayHello() {  
        console.log("Hello");  
    }  
}
```

Looks different.

But internally...

JavaScript converts it into prototype code.

Equivalent to

```
function Person(name) {  
    this.name = name;  
}  
  
Person.prototype.sayHello = function() {  
    console.log("Hello");  
};
```

Same thing!

Proof

```
class Person {  
    sayHello() {}  
}  
  
console.log(typeof Person);
```

Output

```
function
```

Class is actually a function.

Methods Go to Prototype

```
class Person {  
    sayHello() {}  
}
```

Actually becomes

```
Person.prototype.sayHello
```

NOT

```
object.sayHello
```

Verify

```
class Person{  
    sayHello() {}  
}  
  
const p = new Person();  
console.log(p.hasOwnProperty("sayHello"));
```

Output

```
false
```

Because it is stored in prototype.

Constructor in Class

```
class Person{  
    constructor(name) {  
        this.name = name;  
    }  
}
```

```
    }  
}
```

This is simply

```
function Person(name) {  
    this.name = name;  
}
```

extends Keyword

Example

```
class Animal {  
    eat() {  
        console.log("Eating");  
    }  
}  
  
class Dog extends Animal {  
    bark() {  
        console.log("Woof");  
    }  
}  
  
const d = new Dog();  
  
d.eat();  
  
d.bark();
```

Output

Eating

Woof

Internally

Dog.prototype

↓

Animal.prototype

Exactly like `Object.create()`.

super()

```
class Animal{  
    constructor(name){  
        this.name = name;  
    }  
}  
  
class Dog extends Animal{  
    constructor(name){  
        super(name);  
    }  
}
```

`super(name)` **means**

Call parent constructor.

Equivalent to

```
Animal.call(this, name);
```

(Conceptually. In real ES6 classes, `super()` has special behavior.)

instanceof

```
const p = new Person();  
  
console.log(p instanceof Person);
```

Output

```
true
```

Why?

Because JavaScript checks

Does `Person.prototype` exist in `p`'s prototype chain?

Yes.

Return true.

proto

```
const p = new Person();  
console.log(p.__proto__);
```

Output

```
Person.prototype
```

Although `__proto__` exists, avoid using it in modern code. Prefer:

```
Object.getPrototypeOf(p);
```

Prototype vs proto

Many beginners confuse these.

prototype	proto
Property of constructor function	Property of object instance
Used when creating objects	Points to object's prototype
<code>Person.prototype</code>	<code>person.proto</code>

Example

```
function Person() {}  
const p = new Person();  
console.log(Person.prototype === p.__proto__);
```

Output

```
true
```

Complete Memory Diagram

```
class Animal{  
    eat() {}  
}  
  
class Dog extends Animal{  
    bark() {}  
}  
  
const d = new Dog();
```

Memory

```
d  
↓  
Dog.prototype  
↓  
Animal.prototype  
↓  
Object.prototype  
↓  
null
```

Why Prototypes?

Without prototypes

```
1000 objects  
↓  
1000 copies of every method
```

Huge memory waste.

With prototypes

```
1000 objects  
↓
```

ONE shared method

Efficient.

Common Mistakes

✗ Defining methods inside the constructor unnecessarily

```
function Person(name) {  
    this.name = name;  
  
    this.sayHello = function(){};  
}
```

Creates a new function for every instance.

✓ Better

```
Person.prototype.sayHello = function(){};
```

✗ Confusing `prototype` and `__proto__`

`Person.prototype`

is **not** the same thing as

`person.__proto__`

One belongs to the constructor, the other to the created object.

✗ Modifying built-in prototypes

```
Array.prototype.myFunction = function(){};
```

This is generally discouraged because it can conflict with future JavaScript features or other libraries.

Interview Questions

1. What is a prototype?

A prototype is an object from which other objects inherit properties and methods.

2. What is prototypal inheritance?

It's JavaScript's inheritance mechanism where objects inherit directly from other objects through the prototype chain.

3. What is the prototype chain?

A linked chain of prototypes that JavaScript searches when a property or method isn't found on the object itself.

4. What is the difference between `prototype` and `__proto__`?

- `prototype` is a property of constructor functions and is used when creating new instances.
 - `__proto__` (or `Object.getPrototypeOf()`) refers to the prototype of an existing object.
-

5. Are ES6 classes real classes?

No. They provide a cleaner syntax, but under the hood they are built on JavaScript's prototype system.

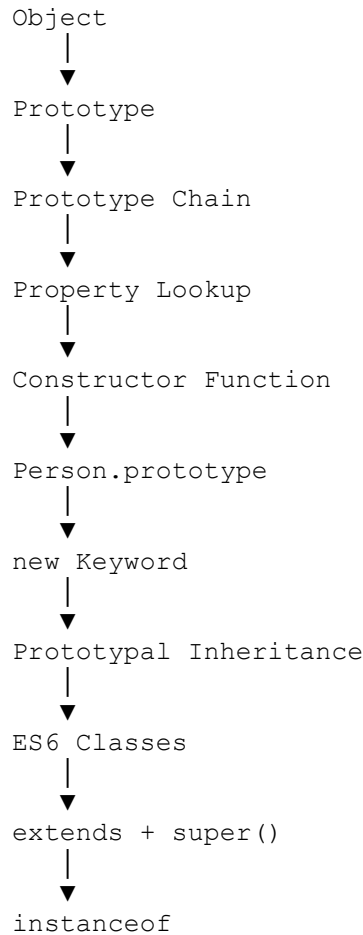
6. Where are class methods stored?

On the class's `prototype`, so all instances share the same methods.

7. What does `extends` do?

It links the child class's prototype to the parent class's prototype, enabling inheritance.

Quick Revision



Key Takeaways

- **Every JavaScript object has a prototype** (except objects explicitly created with `null` as their prototype).
- **JavaScript uses the prototype chain** to look up missing properties and methods.
- **Constructor functions** store shared methods on their `prototype`.
- **ES6 classes are syntactic sugar** over JavaScript's prototype-based inheritance—they don't introduce a new inheritance model.
- **`extends` and `super()`** simplify prototype-based inheritance but still rely on the same underlying mechanism.
- **Methods defined in a class are shared** through the prototype, making them memory-efficient.
- **`instanceof` works by checking the prototype chain**, not by comparing constructors directly.

Understanding prototypes thoroughly will make advanced JavaScript concepts like closures, `this`, object creation, inheritance, and even many framework internals (such as React class components or library APIs) much easier to grasp.

Excellent interview practice task. These three polyfills and the two interview questions are among the **most frequently asked JavaScript interview topics**, especially for frontend and MERN roles.

Part 1: Polyfills

What is a Polyfill?

A **polyfill** is code that adds modern JavaScript functionality to older browsers or environments that don't support it.

For example, if `Array.prototype.map()` didn't exist, we could create our own version.

Think of it like this:

Original JS method is missing ✕

We write our own implementation ✓

Polyfill for `map()`

Original `map()`

```
const numbers = [1, 2, 3];  
  
const result = numbers.map(num => num * 2);  
  
console.log(result);
```

Output

```
[2, 4, 6]
```

Now let's build it ourselves.

Step 1

Add a new function to Array prototype.

```
Array.prototype.myMap = function(callback) {  
  }
```

Inside this function,

this

refers to the array.

Example

```
const arr = [10,20];  
arr.myMap(...)
```

Inside myMap

```
this = [10,20]
```

Step 2

Create a new array.

```
Array.prototype.myMap = function(callback){  
  const result = [];  
}
```

Step 3

Loop over the array.

```
Array.prototype.myMap = function(callback){  
  const result = [];  
  for(let i=0;i<this.length;i++){  
  }  
}
```

Step 4

Call callback.

Remember,

Normal map does this

```
callback(element, index, array)
```

So

```
const value = callback(this[i], i, this);
```

Step 5

Push into result.

```
result.push(value);
```

Step 6

Return result.

Complete Polyfill

```
Array.prototype.myMap = function(callback) {  
  const result = [];  
  for (let i = 0; i < this.length; i++) {  
    const value = callback(this[i], i, this);  
    result.push(value);  
  }  
  return result;  
};
```

Test

```
const numbers = [1,2,3];
```

```
const ans = numbers.myMap((num)=> num * 5);  
console.log(ans);
```

Output

```
[5,10,15]
```

Visual

numbers

↓

```
[1,2,3]
```

↓

Loop

↓

```
callback(1)
```

↓

```
5
```

↓

```
result=[5]
```

↓

```
callback(2)
```

↓

```
10
```

↓

```
result=[5,10]
```

↓

```
callback(3)
```

↓

```
15
```

↓

```
result=[5,10,15]
```

Polyfill for filter()

Original

```
const numbers=[1,2,3,4,5];  
  
const even = numbers.filter(num=>num%2===0);
```

Output

```
[2,4]
```

Notice

filter returns

Only those values whose callback returns

```
true
```

Step 1

```
Array.prototype.myFilter=function(callback){  
  
}
```

Step 2

```
const result=[];
```

Step 3

Loop

```
for(let i=0;i<this.length;i++){  
  
}
```

Step 4

Run callback

```
if(callback(this[i],i,this)){  
  }
```

If true

Push original element

```
result.push(this[i]);
```

Complete Polyfill

```
Array.prototype.myFilter = function(callback){  
  const result=[];  
  for(let i=0;i<this.length;i++){  
    if(callback(this[i],i,this)){  
      result.push(this[i]);  
    }  
  }  
  return result;  
}
```

Test

```
const nums=[10,15,20,25];  
const ans=nums.myFilter(num=>num>15);  
console.log(ans);
```

Output

```
[20,25]
```

Flow

10

↓

callback

↓

false

↓

Ignore

20

↓

callback

↓

true

↓

Store

Polyfill for reduce()

This is the most important one.

Original

```
const nums=[1,2,3,4];  
  
const sum=nums.reduce((acc,current)=>{  
    return acc+current;  
},0);
```

Output

10

Understanding reduce

It keeps one variable

Accumulator

Initially

0

Iteration

acc=0

current=1

↓

1

acc=1

current=2

↓

3

acc=3

current=3

↓

6

acc=6

current=4

↓

10

Step 1

```
Array.prototype.myReduce=function(callback,initialValue) {  
  
}
```

Step 2

Create accumulator.

```
let accumulator=initialValue;
```

Step 3

Loop

```
for(let i=0;i<this.length;i++){  
  }  
}
```

Step 4

Update accumulator.

```
accumulator = callback(accumulator,this[i],i,this);
```

Step 5

Return accumulator.

Complete Polyfill

```
Array.prototype.myReduce=function(callback,initialValue){  
  let accumulator=initialValue;  
  for(let i=0;i<this.length;i++){  
    accumulator = callback(  
      accumulator,  
      this[i],  
      i,  
      this  
    );  
  }  
}
```

```
    return accumulator;
}
```

Test

```
const nums=[1,2,3,4];

const ans=nums.myReduce((acc,current)=>{

    return acc+current;

},0);

console.log(ans);
```

Output

```
10
```

Better Reduce Polyfill (handles missing initialValue)

The native `reduce()` also works when no initial value is provided.

```
Array.prototype.myReduce = function(callback, initialValue) {
    let accumulator = initialValue;
    let startIndex = 0;

    if (accumulator === undefined) {
        if (this.length === 0) {
            throw new TypeError("Reduce of empty array with no initial value");
        }
        accumulator = this[0];
        startIndex = 1;
    }

    for (let i = startIndex; i < this.length; i++) {
        accumulator = callback(accumulator, this[i], i, this);
    }

    return accumulator;
};
```

Example:

```
const nums = [1, 2, 3];

const sum = nums.myReduce((acc, curr) => acc + curr);

console.log(sum);
```

Output:

6

Difference Between map(), filter(), and reduce()

Method	Returns	Callback Return Value	Original Array Changed?
map()	New array	Transformed element	✗No
filter()	New array	true / false	✗No
reduce()	Single value	New accumulator	✗No

Time Complexity

Method Complexity

map	O(n)
filter	O(n)
reduce	O(n)

Part 2: Interview Questions

Q1. What is a Closure?

A **closure** is a function that **remembers the variables from its outer (lexical) scope even after the outer function has finished executing.**

Example

```
function outer() {
  let count = 0;

  return function inner() {
```

```
        count++;
        console.log(count);
    };

    }

const counter = outer();

counter(); // 1
counter(); // 2
counter(); // 3
```

How it works

1. `outer()` is called.
2. `count` is created.
3. `inner()` is returned.
4. Normally, `outer()` would finish and its local variables would disappear.
5. But `inner()` still references `count`, so JavaScript keeps `count` alive.
6. Every call to `counter()` updates the same `count` variable.

Common Uses

- Data privacy (private variables)
- Function factories
- Event handlers
- Timers (`setTimeout`, `setInterval`)
- Memoization
- Module pattern

Interview Definition

A closure is the combination of a function and the lexical environment in which it was created. It allows the function to access variables from its outer scope even after that outer function has returned.

Q2. How Does Prototypal Inheritance Work?

Every JavaScript object has an internal link to another object called its **prototype**.

When you try to access a property:

1. JavaScript looks on the object itself.
2. If not found, it looks on the object's prototype.
3. It continues up the **prototype chain** until it finds the property or reaches `null`.

Example

```
const animal = {
  eat() {
    console.log("Eating...");
  }
};

const dog = Object.create(animal);

dog.bark = function () {
  console.log("Barking...");
};

dog.bark(); // Barking...
dog.eat();  // Eating...
```

Prototype Chain

```
dog
|
▼
animal
|
▼
Object.prototype
|
▼
null
```

Constructor Function Example

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function () {
  console.log(`Hello, I'm ${this.name}`);
};

const p1 = new Person("Alice");

p1.sayHello();
```

Here:

- name is stored on the instance (p1).
- sayHello() is stored once on Person.prototype.
- All Person instances share the same method, making it memory-efficient.

Why use prototypes?

Without prototypes:

```
function Person(name) {  
  this.name = name;  
  this.sayHello = function () {  
    console.log(`Hello, I'm ${this.name}`);  
  };  
}
```

Every object gets its own copy of `sayHello()`, wasting memory.

With prototypes, one shared method serves all instances.

Interview Definition

Prototypal inheritance is JavaScript's inheritance model where objects inherit properties and methods from other objects through the prototype chain. When a property isn't found on an object, JavaScript searches its prototype and continues up the chain until the property is found or the chain ends.

Interview Tips

- **Polyfills**
 - Explain that you're recreating native methods using `Array.prototype`.
 - Mention callback signatures (`element, index, array`) and edge cases (like `reduce` without an initial value).
- **Closures**
 - Focus on "function + lexical environment."
 - Be ready to explain practical uses such as private state and callbacks.
- **Prototypal Inheritance**
 - Emphasize the prototype chain and property lookup.
 - Explain why shared methods on prototypes are more memory-efficient than defining methods inside constructors.